Lab 1A: The Agentic Build Loop

Duration: 35 min

After: Installation & Agentic Loop slides (Day 1, Block 1)

Prerequisites: Lab 0 complete (Claude answered a test question; model checked with `/model`)

Objective

Run your first full agentic loop: describe a system, watch Claude Code plan -> execute -> validate -> self-heal, then use checkpoints and rewind to undo a change safely. You are not learning to prompt an AI - you are learning to manage an AI engineer.

Deliverable (toolkit chain 1 of 12)

A running **business-dashboard** repo with a first git commit. This repo is the vehicle for your personal toolkit: over the next 11 labs you will add a CLAUDE.md, two skills, hooks, and a validated plugin manifest to it.

> ***START MARKER:*** *Terminal open,* `claude --version` *returns 2.1.x+.*

Before you start: you'll be approving a lot in this lab

> ***You will be approving a lot of actions in this lab. That's deliberate.***
>
> *Claude Code ships asking permission for everything, and that is the correct default for a tool you have not configured yet. Today you run it that way on purpose.*
>
> ***Notice how it feels.*** *Count the clicks. Notice when you stop reading the prompts and start reflex-approving. That feeling has a name - approval fatigue - and it is the single most common reason people either abandon the tool or turn off every guardrail at once.*
>
> ***We fix it in Block 4*** *(Lab 1D), where you write permission rules that pre-approve the safe things and keep asking about the dangerous ones. Do not go looking for that fix now. The friction is the lesson.*

1. Pre-Flight Check (2 min)

```
claude --version # 2.1.x+ - installed natively in Lab 0, no npm/Node needed
git --version # 2.x.x
node -v # 18+ - required for THIS LAB'S PROJECT only, not for Claude Code
```

If any check fails, go back to Lab 0 recovery.

2. Create the Project and Start Claude (2 min)

```
mkdir business-dashboard
cd business-dashboard
claude
```

Confirm the model:

```
/model
```

Expected output marker: `/model` shows the account/provider default. Defaults vary. Select Sonnet 5 for normal implementation; escalate to Opus 5 for complex reasoning when available and justified.

> ***Quality-of-life:*** *try* `/btw <question>` *any time for a side question that does not pollute your context. Need shell output in the conversation? Prefix with* `!` *- e.g.* `!ls` *- and Claude responds to the output directly without a second prompt.*

3. The Build Prompt (10 min)

Copy and paste this prompt exactly:

```
Initialize this as a git repo and build me a business dashboard web application:

Tech stack: Node.js, Express, SQLite (via better-sqlite3), vanilla HTML/CSS/JS frontend
No external CSS frameworks - write clean, modern CSS

Database tables:
1. metrics - id, name, value (number), category, recorded_at
2. tasks - id, title, status (pending/in-progress/completed), priority (high/medium/low),
assigned_to, created_at, completed_at

API endpoints:
- GET /api/metrics (filter by ?category=)
- POST /api/metrics
- GET /api/tasks (filter by ?status= and ?priority=)
- POST /api/tasks
- PUT /api/tasks/:id
- DELETE /api/tasks/:id
- GET /api/dashboard (summary: all metrics + task counts by status + high priority tasks)

Frontend: Dark theme dashboard at / showing metrics as responsive cards and tasks as an
interactive list with status badges and priority indicators.
Seed the database with 8 sample metrics across categories (financial, customers, operations,
hr) and 5 sample tasks.

Initialize npm, install dependencies, create everything, and start the server.
Use port 3100 (or PORT env var).
Include a .gitignore for node_modules and *.db files.
```

Watch the loop:

Phase	What Claude does
Plan	Analyzes requirements, decides file structure
Execute	Creates files, runs npm install, writes code
Validate	Checks for errors, starts and tests the server
Self-heal	Reads any error, diagnoses, fixes, re-runs - no help needed

Approve permission prompts as they appear (or "Yes, and don't ask again"). Do NOT type while Claude is working. Pauses of 10-30 seconds are normal reasoning time.

Guardrail - don't let approvals eat your lab time

> ***If you're more than 10 minutes into this step and still approving every single file write, press Shift+Tab once to enter accept-edits mode and keep going.***

> *That is the escape hatch, not the lesson. It stops the clicking so you can finish the build.* ***We'll explain exactly what that did in Block 4*** *- which modes exist, what each one still asks about, and how to set the boundary once instead of answering it a hundred times.*
>
> *Before you press it: make a mental note of roughly how many approvals you got through. You'll be asked.*

Expected output marker: Claude reports the server running on port 3100.

4. See It in the Browser (2 min)

Open http://localhost:3100

Expected output marker: A dark-themed dashboard with metric cards and a task list.

5. Test the System Through Claude (4 min)

```
Test the API: add a new metric called "New Leads This Week" with value 42 in the "sales"
category. Then add a task "Review Q2 projections" with high priority assigned to me. Then
complete task 1. Show me the results after each operation.
```

Refresh the browser - the new data appears.

6. First Commit (2 min)

```
Commit everything with the message "feat: initial business dashboard with API and UI"
```

If Claude asks to use a commit skill, select Yes - you will learn Skills on Day 2.

7. Checkpoints & Rewind

> ***Surface note:*** *these keyboard steps assume* `claude` *is running in the* ***VS Code integrated terminal*** *(our classroom setup from Lab 0). If you are in the extension's chat panel instead, use its own mode switcher and rewind button - same capability, different control.*

- Your Undo Button (8 min)

Claude Code checkpoints your session automatically. Let's prove you can always get back.

Step 7a - make a change you'll regret:

```
Change the entire dashboard theme to bright pink with yellow text.
```

Wait for it to finish, refresh the browser. Admire the horror.

Step 7b - rewind:

Double-tap **Esc** (press Escape twice quickly).

In the VS Code panel (our setup): hover the message you want to go back to and click the

rewind arrow in its top-right corner. Three choices appear - these are the exact labels you will see:

Panel label	What it does	Terminal equivalent
Fork conversation from here	Branches the chat from this point; files untouched	conversation only
Rewind code to here	Reverts the files; chat history stays	code only

Panel label	What it does	Terminal equivalent
Fork conversation and rewind code	Both - the full undo	both

In the terminal: double-tap `Esc` for the same three scopes under shorter names.

Expected output marker: The rewind options appear with those three labels.

- **both** (full undo)

Step 7c: Select the checkpoint from before the pink theme and choose **Fork conversation and rewind code**. Refresh the browser - dark theme is back.

```
!git status
```

Expected output marker: Working tree clean (or only your pre-pink state). Rewind restored the files.

> ***Why this matters:*** *rewind is ALWAYS better than arguing with a contaminated context. Lab 1C is built entirely around this muscle.*

8. Reverse Engineer (3 min)

```
Explain the architectural decisions you made while building this system.
```

Then challenge it:

```
Identify the 3 biggest architectural weaknesses in this project. Rank them by risk if this
system went to production. What would break first under scale?
```

9. Vague vs. Specific (2 min)

Try: `Add validation.` - watch Claude guess. Then:

```
In @server.js, modify only the POST /api/tasks route. Validate:
- title: non-empty string, max 200 characters
- priority: one of "high", "medium", "low"
- assigned_to: optional, but if provided must be a non-empty string
Return 400 with { error: "description of what's wrong" } for invalid requests.
```

Then: `Which instructions in my prompt changed your implementation?`

> *Specificity isn't extra work - it's less work. One precise prompt beats three vague ones.*

FINISH MARKER - Success Checklist

- [ ] Express server running on port 3100
- [ ] Working dark UI at http://localhost:3100
- [ ] First git commit made
- [ ] Rewind menu opened with double-Esc and restored a checkpoint
- [ ] Input validation on POST /api/tasks
- [ ] Sonnet 5 confirmed as your model

If Stuck

Problem	Recovery
Claude stops mid-build	Type `exit`, then `claude`, then: "The build stopped before finishing. Continue building the dashboard application."
Port 3100 in use	"Port 3100 is in use. Switch to port 3456." (If Claude finds another app on 3100, it will ASK before touching it - that is correct behavior, answer it)
`gyp ERR! find VS` during npm install	Expected on some Windows/Node combos. Watch Claude diagnose and pin a working version itself - that IS the loop. If it stalls: "better-sqlite3 has no prebuilt binary for my Node version. Pin a version that does, or use Node's built-in node:sqlite instead."
Git warns `LF will be replaced by CRLF`	Normal on Windows. Ignore
`node` not found	Node is for the lab project only - install LTS from nodejs.org
No rewind arrow on the message	Hover directly over the message bubble - the arrow sits in its top-right corner. In the terminal: press Esc twice QUICKLY, or type `/rewind`
Pink theme survives rewind	You chose **Fork conversation from here**, which leaves files alone. Rewind again and choose **Fork conversation and rewind code**

Debrief Questions

1. Where in the loop did Claude self-heal without your help?
2. What did rewind offer beyond git - and when would you fork the conversation, rewind the code, or do both? (Note the word *fork*: you are branching from an earlier point, not deleting history.)
3. What one sentence in the specific validation prompt saved you a review cycle?
4. **Roughly how many times did you approve an action in this lab? What would you have wanted instead?** (Be specific: which prompts would you never want to see again, and which ones do you still want stopping you?) Hold onto your answer - we build exactly that in Block 4.